次元造物 · 技术文档

> 版本：v1.0 | 日期：2026-10-21 | 赛道：第九届传智杯 · AI创新应用挑战赛

---

1. 项目概述

「次元造物」是一个面向二次元角色的换装与 3D 化工具。用户上传角色图、输入一句服装描述，系统在约 30 秒内给出**同一描述下的三张候选**（单张约 10 秒，三张并行）；选定一张后可继续微调，再点击生成，约 120 秒后得到可在浏览器中 360° 旋转的 3D 模型（四视图重建链路）。

项目以「无服务器 + 双 AI 服务编排」为核心，全部功能跑在腾讯 EdgeOne Pages 上，无独立服务器、无数据库。

1.1 设计目标

目标	具体约束
---	----------
可用	任一外部服务故障时，页面不得白屏，必须给出可读原因
快	首屏可交互 < 1s；单张 2D < 15s；三候选 < 35s（并行）
便携	单机可复现，本地与线上共用同一份核心代码
安全	API Key 绝不进入前端与版本库
可维护	无构建步骤，改一行即可上线

1.2 不在范围内

用户体系、云端同步（作品集仅存于浏览器本地）、多语言、移动端原生 App。

三者中**只有「作品持久化」做了一半**：本版用 IndexedDB 实现了本地作品集（最多 10 条、可恢复可再编辑），

但没有账号与云端，换设备即失去。这是有意为之——先验证"要不要反复用"，再决定要不要上云。

---

2. 意图控制策略（Prompt 与上下文设计）

本赛道考的不是"能不能让 AI 写代码"，而是**人能不能把意图稳定地灌进 AI**。这一章写清三件事：

给模型的控制、给生成模型的控制、给 AI 编程助手的控制。

2.1 三层控制，各管一段

层	控制对象	手段	落点
生成层	图像模型（万相）	固定不变项 + 用户只填变化项	`_lib/wan.js` 的 `SYSTEM_CONTEXT`
交互层	用户意图	三候选取自采样，而非替用户改 prompt	`index.html` 的 `runTriple`
工程层	AI 编程助手	用可验证的约束代替"随便改"	提交历史 + 本文档各章

2.2 生成层：把"不变"写死，把"变"交给用户

换装最常见的失败不是画得丑，而是**换了衣服、换了脸。**所以提示词被拆成两段：

```
// _lib/wan.js
const SYSTEM_CONTEXT =
  '保持图中二次元虚拟角色的面部特征、发型、瞳色和整体画风完全不变，' +
  '仅根据以下描述更换服装。输出高质量、细节丰富的二次元插画风格。';

text: `${SYSTEM_CONTEXT} 换装描述：${text}`
```

- **前半段由程序掌握**：面部、发型、瞳色、画风——四项不变项，用户改不了，也不该改。
- **后半段交给用户**：只描述衣服。

为什么这样切：如果把"保持角色一致"也交给用户去写，他就得每次重复一遍，且漏写一次就换脸。

把不变项固化进代码，用户输入的语义空间就从"画一张图"收缩到"换一套衣服"——意图被约束，

结果才稳定。这是本项目的第一个意图控制决策，也是最有效的一个。

2.3 交互层：多候选靠采样，不靠改 prompt

早期版本我让程序注入三套不同风格的提示词（古风/赛博/礼服），结果是**程序替用户改了风格。**

改成同一段描述并行跑三次后：

做法	差异来源	谁在决定风格
注入三种预设 prompt	程序改写文本	程序（错）
同 prompt 并行三次	模型采样随机性	用户写的描述（对）

判断标准可以写死成一句话：**多候选之间的差异，必须来自模型的随机性，不能来自程序替换提示词。**

一旦程序改 prompt，"三候选"就从"帮用户表达"退化成"替用户决定"。

微调同理：四个微调入口只做两件事——把当前作品设为底图、把约束句写进输入框，

不发起请求。因为微调是最容易误触的操作，而每次生成都要花钱，

把"改描述"和"花钱生成"拆成两步，等于把成本控制做进了交互。

2.4 工程层：给 AI 的约束必须是可验证的

对 AI 编程助手，我不给"写一个换装应用"这种目标级指令，而给**能被实测证伪的约束**：

约束	为什么这么定	验证方式
不换技术栈、前端保持单文件	保证部署链路不变，改动可回滚	`public/` 始终只有 1 个 html
后端必须能在云函数单请求内跑完（≤60s）	平台硬限制，超时即失败	异步轮询方案被此约束否决，改同步调用
失败必须可见、可重试	不许用演示模型冒充结果	接口返回 `success:false`，前端拒绝呈现 demo
每次改动先说明依据，改完给实测数据	防"看起来对"的推断	5MB 模型对照实验推翻了"模型太大"的误判

这套约束的价值在于可证伪：每一条都能用一次操作或一个数字打假，所以 AI 无法用"看起来完成了"糊过去。

本项目被推翻的三次判断（模型太大 / 组件是懒加载 / 异步更快）全部来自这种可测量的对照。

2.5 意图控制的代价

这套策略不是免费的：

- 不变项写死，意味着**用户想改画风时必须换一个产品入口**（本版不支持）
- 三候选并行，意味着**三分之二的 2D 成本沉没**——换来的是"在决定前看见三种可能"
- 约束越硬，AI 能自由发挥的空间越小，**探索性功能只能靠人先想清楚再加**

取舍结论：这个产品的核心价值是"快速试错"，所以稳定性和可预期优先于自由度。

---

3. 场景与用户价值

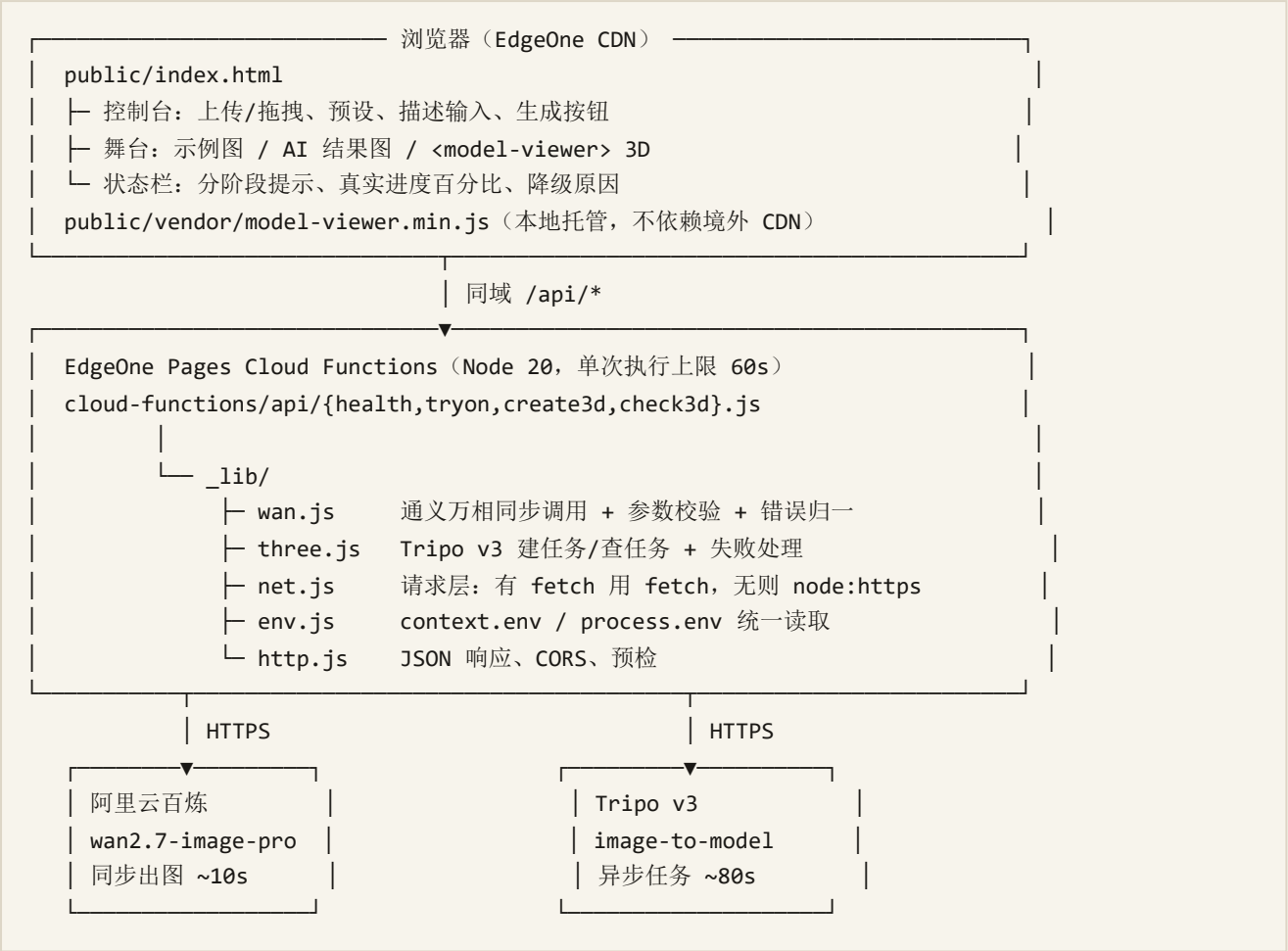
传统「给角色换套衣服」的路径是：找画师 → 描述需求 → 等稿 → 改稿，成本数百至数千元、周期数天。

本工具把单次试错成本压到约 0.5 元 / 10 秒，把 3D 化的边际成本压到约 0.3 美元 / 80 秒。价值不在「生成一张图」，而在**让试错变得几乎免费**：用户可以连续试十套造型，再决定让画师精细绘制哪一套。

目标用户：同人创作者、独立游戏/漫剧前期概念设计、虚拟主播形象迭代。

---

4. 总体架构



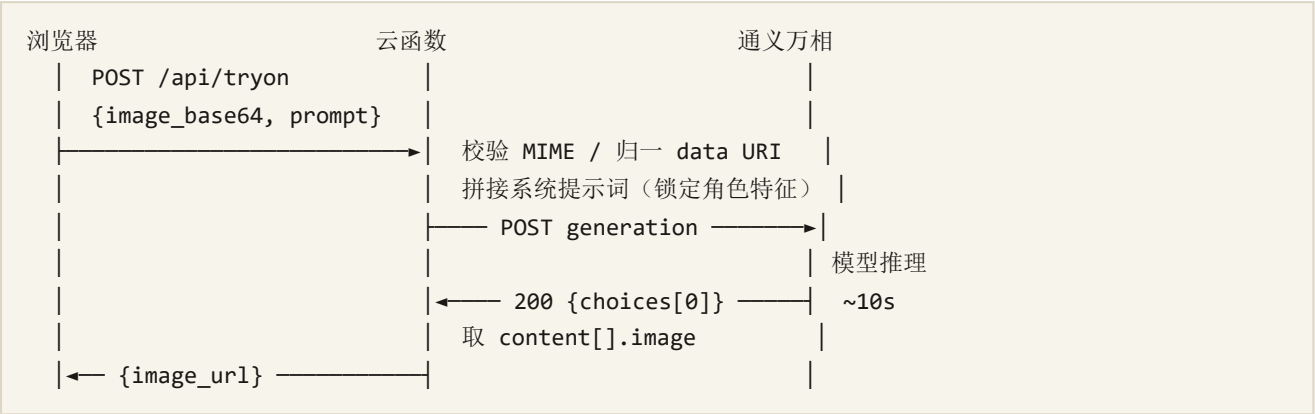
4.1 为什么是「同域云函数」

- 若前端静态托管、后端另置，需要处理跨域与两套域名，且本地调试链路不同
- EdgeOne 的云函数与静态资源同域，前端只需相对路径 `/api/*`
- 本地用 Express 复刻同样的路由路径，**同一份 `_lib` 在两端运行**，本地通过即线上大概率通过

---

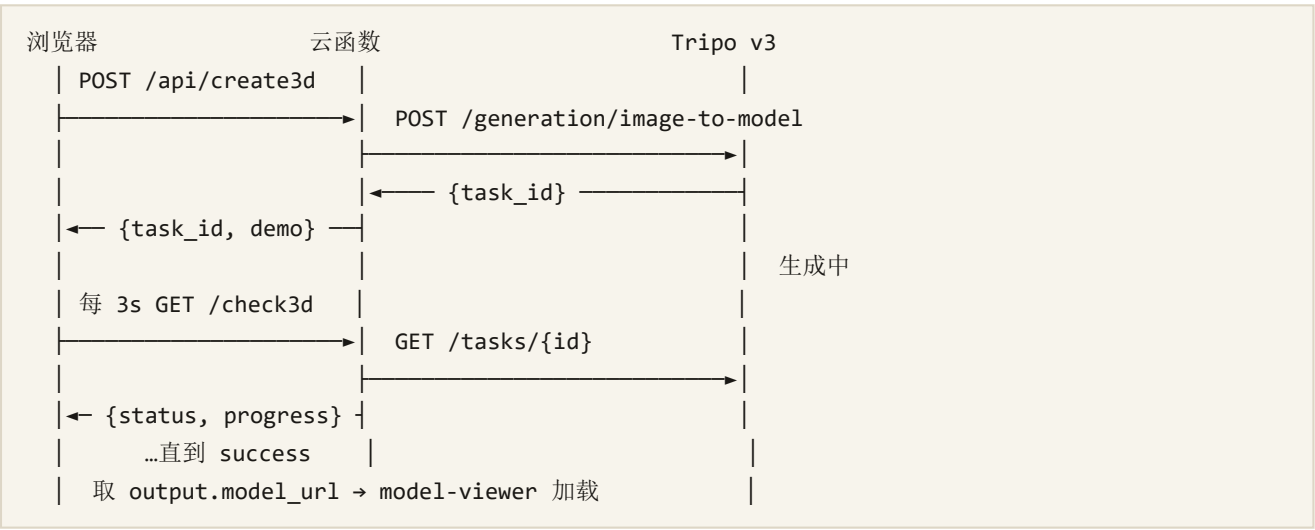
5. 关键流程

5.1 2D 换装（同步）



为什么同步：万相的异步模式需要「建任务 → 每 3 秒轮询」，最长 150 秒，超出云函数 60 秒上限。同步模式一次请求拿图，实测 10 秒，比异步（16 秒）更快且少 N 次请求。

5.2 3D 生成（异步 + 前端轮询）



注意：Tripo 的输出链接仅 5 分钟有效，因此模型必须在生成成功后立刻加载或下载。

---

6. 技术选型与取舍

6.1 前端：原生单文件 vs 框架

方案	结论
原生单文件	✅ 无构建、EdgeOne 直接托管、改一行即上线、评委可读源码

代价：样式与逻辑内联，文件较长（约 1300 行）。通过区块注释分区缓解。

6.2 3D 渲染：model-viewer vs Three.js

`model-viewer` 是 Google 的 Web Component，一行标签即可获得：轨道控制、自动旋转、环境光照、进度事件、GLB 解码。

自研 Three.js 需处理加载器、相机、光照、控制、色调映射，代码量与故障面都显著更大。**本项目 3D 只是展示环节，不值得自研。**

6.3 图像模型：wan2.7-image-pro

- 支持「图 + 文」多模态输入，天然适配"保留角色特征、只换服装"
- 同步接口一次出图，便于在云函数里跑
- 输出 1K PNG 约 1.6MB、2K 约 5.4MB，可通过 `WAN_SIZE` 调节

6.4 3D 模型：Tripo v3 vs v2

	v2	v3
----	----------	----------
图片传入	必须先 `POST /upload` 拿 `image_token`（multipart）	**直接传公网 URL**
依赖	需要 fetch + FormData	只需 JSON POST
结论	在裁剪过的运行时里跑不通	✅ 采用

v3 的 `face_limit` 参数可以把 139 万面压到 3 万面（体积从 42 MB 降到 4.98 MB），且不加价。

这个参数经历过一次决策反复，过程比结论更有价值：

- 最初因为 42 MB 模型在浏览器里转不动，先设 `face_limit=30000` 保可用性；
- 后来发现"转不动"的真正原因是**本机代理导致大文件下载极慢**，与面数无关——用 5 MB 的模型在同样条件下也触发了同样的超时；
- 关掉代理后实测全精度模型可正常加载，于是改回全精度，同时把加载超时从 60 秒放宽到 90 秒，并给下载缓存加 12 MB 体积守卫（超过则不驻留内存，改为点击时再取）。

最终配置：全精度（约 145 万面 / 40 MB），弱网仍可能较慢，此时会明确提示而不是静默降级。

---

7. 核心实现

7.1 请求层双通道（net.js）

部分 Serverless 运行时提供 `Request`/`Response` 但没有全局 `fetch`（本项目线上曾实测报 `fetch is not defined`）。因此请求层做成双通道：

```
const send = typeof fetch === 'function' ? viaFetch : viaHttps;
```

- 有 `fetch`：走标准 `fetch` + `AbortSignal.timeout`

- 无 `fetch`：走 `node:https`，手写超时与 JSON 解析，超时统一抛 `TimeoutError`，上层逻辑无需分支

设计要点：让修复在"两种假设"下都成立，不赌根因。

7.2 3D 任务无状态化

云函数是分布式、无共享内存的。若把任务状态存在进程内 `Map`，换个节点就读不到。

方案：`task_id = t3d_<base36(时间戳)>`，查询时解析时间戳，`elapsed >= 2000ms` 即视为完成。任何节点算出同一结果。

副作用：无法保存真实模型中间态。因此该方案**仅用于演示降级路径**，真实 3D 走 Tripo 的 task_id。

7.3 失败处理：可见地失败，而不是假装成功

2D 与 3D 两段的外部依赖不同，失败策略也不同：

层级	触发条件	用户可见结果
L1	未配置密钥	明确提示"服务端未配置 ALIYUN_API_KEY / TRIPO_API_KEY"，不发起请求
L2	接口返回非 0 码（如 2010 余额不足）	提示具体错误码与余额，3D 段直接失败
L3	请求超时 / 网络不可达	提示底层错误（如 ENOTFOUND），允许重试
L4	模型已生成但浏览器加载失败	保留换装图，90 秒后提示加载失败并提供重试

3D 段的特殊约定：服务端在 Tripo 不可用时会返回一个降级任务占位，但**前端把它当作失败处理**（`if (checkData.demo) throw`），不会把演示模型当成用户作品展示。用户看到的是"当前不可用 + 原因 + 可重试"，而不是一个"看起来成功了"的假结果。

统一原则：失败必须可见、可重试。宁可显示"当前不可用 + 原因"，也不能让界面无限转圈，更不能用一个演示模型冒充真实结果。

7.4 前端状态与体验

- **分阶段提示**：把 10 秒的等待拆成"解析参考图 → 理解描述 → 生成中"等多段，降低感知等待
- **真实进度**：3D 阶段读取 `check3d` 返回的 `progress`；模型下载阶段监听 `model-viewer` 的 `progress` 事件
- **超时兜底**：90 秒未 `load` 判定为失败，保留换装图并说明原因（全精度模型约 40 MB，阈值过短会误杀）
- **组件守卫**：`ensureModelViewerReady` 在 8 秒内轮询 `customElements.get('model-viewer')`，避免自定义元素未注册导致 `src` 设置完全无效

7.5 本地持久化（IndexedDB）

生成结果默认是临时链接（图片 24 小时、模型 5 分钟），刷新即失效。为了让"试过的造型"可回溯，

前端用 IndexedDB 做本地作品集：

项	实现
数据库 / 表	`ciyuan-atelier` / `generations`（`keyPath: 'id'`，索引 `createdAt`）
容量策略	最多 10 条，写入后按 `createdAt` 倒序裁剪，超出即删除最旧记录
保存内容	底图、描述词、三个候选方案（Blob）、创建时间；**3D 模型不保存**（见下方体积守卫）
可做的事	查看、恢复、再次编辑、删除
刷新之后	底图、描述、候选均可恢复；**3D 模型需重新生成**（约 40MB，超出 12MB 守卫，不入库）

为什么选 IndexedDB 而不是 localStorage：模型是二进制大对象，localStorage 只能存字符串且上限约 5 MB；

IndexedDB 可直接存 Blob，容量按配额走。

体积守卫（两处，均以 12 MB 为界）：

对象	行为
3D 模型	`prefetchModelBlob` 先发 `HEAD` 取 `content-length`，超过 12 MB 就不驻留内存，改为点击下载时再取
候选图	`cacheHistoryCandidates` 同样先 `HEAD` 判断，单张超限则只保留临时 URL、不转存 Blob

单条记录约 3×5.4 MB（2K 图）≈ 16 MB，10 条约 160 MB，落在浏览器配额的安全范围内。

为什么模型一律不入库：全精度模型约 40 MB，是候选图之和的两倍多，且 Tripo 的输出链接只有 5 分钟有效。

把它存进 IndexedDB 会让单条记录涨到 56 MB、10 条直接上 1.1 GB —— 手机浏览器必然配额失败。

所以选择不存，代价是刷新后 3D 需要重新生成（约 100 秒、约 30 credits）。

界面如实反映这一点：恢复历史时若没有缓存的模型 Blob，历史卡片标记「3D 已过期」，

按钮变为「重新生成 3D」，下载入口保持禁用——不拿已过期的链接冒充"已生成"。

刻意不做的事：不做云端同步、不做账号体系。历史只属于当前浏览器，换设备或清缓存即丢失——

这一点在界面上明说，避免用户误以为存在云端。

7.6 二次微调：只载入，不生成

选定作品后提供四个微调入口：仅改配色、仅改材质、减少配饰、自定义微调。

关键在于**它们不发起请求**：

- 把当前选定的图载入为新底图
- 把对应的约束描述写进输入框（如"保持面部、发型、体型、版型一致，只调整服装配色"）
- 状态栏提示"已载入，确认后主动点击开始换装，本操作尚未生成新图"

为什么这样设计：微调是最容易被误触的操作，而每次换装都要花钱。把"改描述"和"花钱生成"拆成两步，用户可以先看清描述再决定。这是把成本控制做进交互，而不是只写在文档里。

7.7 真实结果与主动下载

- **没有真实模型时禁止下载**：`downloadModel` 前置校验 `state.modelIsReal`，演示模型不会被导出
- **角色卡需主动点击**：用 Canvas 合成卡片（作品 + 描述 + 水印），用户点击才绘制并下载
- **任何文件都不自动下载**：图片、角色卡、模型全部由用户主动触发

原因：自动下载在演示场景里是干扰（浏览器弹出下载栏、文件堆在下载目录），

更重要的是——它会让人分不清"这是真结果还是兜底产物"。主动下载把判断权交回用户。

7.8 输入校验与接口收敛

位置	校验
前端上传	仅 JPG / PNG / WEBP；单张 ≤ 12 MB；类型与体积在读取前拦截
后端换装接口	校验 data URL 的 MIME；描述词必须为非空字符串且 ≤ 1000 字符
描述输入框	`maxlength="1000"`，与后端上限一致

健康检查接口做了收敛：上线初期 `/api/health` 会返回模型名、服务商地址、密钥长度、Node 版本、运行时能力等排障信息；这些信息对前端无用，却等于把内部结构公开给任何访问者。

现在只返回四项：`ok`、`api_key_configured`、`tripo_key_configured`、`time`。

---

8. 部署与运维

8.1 部署拓扑

单项目、单域名：静态资源走 CDN，`/api/*` 走云函数。加速区域选「全球（不含中国大陆）」，免 ICP 备案。

8.2 配置项

变量	作用	缺省
----------	--------	----------
`ALIYUN_API_KEY`	百炼鉴权	无（2D 不可用）
`TRIPO_API_KEY`	Tripo 鉴权	无（3D 降级）
`TRIPO_FACE_LIMIT`	输出面数上限	不限制
`WAN_SIZE`	输出尺寸	`2K`
`DEMO_MODEL_URL`	降级模型	`/models/robot.glb`

8.3 自检接口

`GET /api/health` 只返回四项：`ok`、`api_key_configured`、`tripo_key_configured`、`time`。

部署后先看这个接口，确认两个密钥都已生效，再点页面——这是排障最快的一步。

早期版本会返回模型名、服务商地址、密钥长度与运行时能力，排障方便但等于公开内部结构，现已全部收敛。排障细节改为只在服务端日志输出。

8.4 已知运维注意事项

- 云函数配置变更后需重新部署才生效
- 控制台"已部署"与边缘"已生效"存在时间差，验证需带缓存破坏参数（`?v=xxx`）
- 输出目录必须是 `public`，否则根目录无 `index.html` 会 404

---

9. 性能与成本

9.1 实测数据

环节	耗时 / 体积
2D 单张换装（同步调用）	10 s，输出 1.6MB（1K）/ 5.4MB（2K）
2D 三候选（并行三次）	约 30 s 出齐（单张并行，不是三倍串行）
四视图扩图 `image-to-multiview`	21 s
3D 重建 `multiview-to-model`（当前默认）	98 s，40.7MB，145 万面
3D 单图重建 `image-to-model`（早期实测）	86 s，42.1MB，139 万面
3D 单图 + `face_limit=30000`（弱网方案）	80 s，**4.98MB**，29,782 面
演示模型加载（本地 464KB）	0.011 s

> 面数随输入不同在 139–145 万之间浮动，体积约 40–42 MB。完整链路（三候选 + 四视图重建）

> 端到端约 130 秒。

9.2 成本

项	单价
万相出图	约 0.5 元/张（一次三候选 ≈ 1.5 元）
Tripo 四视图扩图 `image-to-multiview`	约 10 credits
Tripo 3D 重建（四视图 / 单图）	20–30 credits
完整 3D（四视图，当前默认）	约 30–40 credits ≈ 0.3–0.4 美元/次
托管	EdgeOne 免费额度（100 万次函数调用/月）

> 成本上最关键的一条：**三张候选只做 2D 比较，3D 只跑选中的那一张**——

> 若三张都建模，单次交互成本会变成三倍。

9.3 一个反直觉的结论

模型体积主要不是贴图造成的。42MB 的模型里贴图仅 4.17MB，其余约 38MB 全是网格数据（72 万顶点 / 139 万三角面）。因此**限面数比降贴图质量有效得多**——这条结论在技术上成立，但最终产品没有采用限面数方案，

因为后来查明"加载慢"的真实原因是本机代理导致大文件下载受阻，与面数无关。最终选择保留全精度，用放宽超时（90 秒）与下载缓存体积守卫来兜底（详见 6.4 与 7.3）。

---

10. 问题复盘

现象	初始猜测	实测结论	修复
----------	---	----------	----------
首页 404	代码没推上去	远端已是最新，`index.html` 已移入 `public/`，输出目录仍是根	改输出目录
`fetch is not defined`	Node 版本过低	运行时为 v20.19.3 且有 fetch，但请求路径仍失败	加 `node:https` 双通道
3D 一直在"载入中"	模型太大	5MB 同样不触发 `load`	组件本地化 + 取消 `display:none` + 8 秒守卫
42MB 模型转不动	模型面数过高	真正原因是本机代理导致大文件下载极慢，5MB 模型同样超时	关代理后改回全精度；超时 60→90s，缓存加 12MB 守卫
部署后配置不生效	变量拼错	构建未完成 / 缓存	带 `?v=` 重查 health

方法论：每个"看起来合理"的解释，都要用一个可测量的量去证伪。

---

11. 局限与演进

11.1 当前局限

- 3D 生成受第三方速度制约，现场演示需预留 90 秒
- 模型与图片均为临时链接（5 分钟 / 24 小时），未做持久化
- 历史仅存于浏览器本地（IndexedDB，最多 10 条），换设备或清缓存即失去
- 全精度模型约 40 MB、145 万面，弱网加载慢，且不适合移动端长时间展示

11.2 演进方向

- **产物持久化**：当前作品集只存在浏览器本地，下一步转存对象存储，实现跨设备作品集
- **异步化体验**：3D 生成期间允许用户继续操作，完成后通知
- **提示词工程**：把"换装描述"做成风格模板库，降低输入门槛
- **多视图重建**：用正面 + 侧面两张图生成更准确的角色模型
- **导出格式**：支持 FBX/OBJ/USDZ，直接进 Unity/Blender

附：文档与代码索引

内容	位置
使用与复现指南	`README.md`
技术文档	`TECHNICAL.md`（本文件）
一键启动（Windows）	`start.bat`
前端（结构 + 样式 + 交互）	`public/index.html`
云函数入口	`cloud-functions/api/`
共享核心	`cloud-functions/_lib/`
运行配置	`edgeone.json`
环境变量	`.env`